

B6: Declarative Rules Hardening (<10 committed rows)

Query

Query History

Scratch Pad X

```
1  --Step 1 -- Create / modify tables with declarative constraints
2  -- Create Project table
3  DROP TABLE IF EXISTS project CASCADE;
4  CREATE TABLE project (
5      project_id SERIAL PRIMARY KEY,
6      project_name TEXT NOT NULL,
7      start_date DATE NOT NULL,
8      end_date DATE NOT NULL,
9      budget NUMERIC(10,2) NOT NULL CHECK (budget > 0),
10     status TEXT NOT NULL CHECK (status IN ('Active', 'Completed', 'On
```

Data Output

Messages

Notifications

NOTICE: drop cascades to constraint expense_project_id_fkey on table expense

CREATE TABLE

Query returned successfully in 123 msec.

Query

Query History

Scratch Pad X

```
1  -- Create Expense table
2  DROP TABLE IF EXISTS expense CASCADE;
3  CREATE TABLE expense (
4      expense_id SERIAL PRIMARY KEY,
5      project_id INT NOT NULL REFERENCES project(project_id) ON DELETE
6      expense_date DATE NOT NULL,
7      amount NUMERIC(10,2) NOT NULL CHECK (amount > 0),
8      category TEXT NOT NULL CHECK (category IN ('Travel', 'Supplies',
9      CHECK (expense_date >= '2020-01-01')
10 );
```

Data Output

Messages

Notifications

CREATE TABLE

Query returned successfully in 132 msec.

Query

Query History

Scratch Pad X

```
3  INSERT INTO project (project_name, start_date, end_date, budget, status)
4  VALUES
5  ('Road Upgrade', '2024-01-01', '2024-12-31', 50000, 'Active'),
6  ('Health Center', '2024-03-01', '2025-03-01', 120000, 'Completed');
7
8  INSERT INTO expense (project_id, expense_date, amount, category)
9  VALUES
10 (1, '2024-02-01', 3000, 'Labor'),
11 (2, '2024-05-01', 2500, 'Supplies');
12
```

Data Output

Messages

Notifications

INSERT 0 2

Query returned successfully in 133 msec.

Query

Query History

1

--Step 3 – Failing Inserts (wrapped in a transaction and rolled back)

2

BEGIN;

3

4

-- Violates CHECK (negative budget)

5

INSERT INTO project (project_name, start_date, end_date, budget, stat

6

VALUES ('Invalid Project', '2024-01-01', '2024-12-31', -10000, 'Activ

7

8

-- Violates date order (end_date < start_date)

9

INSERT INTO project (project_name, start_date, end_date, budget, stat

10

VALUES ('Wrong Dates', '2025-06-01', '2025-05-01', 20000, 'Active');

11

12

-- Violates CHECK (negative expense amount)

13

INSERT INTO expense (project_id, expense_date, amount, category)

14

VALUES (1, '2024-03-01', -500, 'Travel');

15

16

-- Violates CHECK (invalid category)

17

INSERT INTO expense (project_id, expense_date, amount, category)

18

VALUES (2, '2024-03-01', 1000, 'Food');

19

20

-- Rollback to prevent bad data from being saved

21

ROLLBACK;

Data Output

Messages

Notifications

ERROR: new row for relation "project" violates check constraint "project_budget_check"

Failing row contains (3, Invalid Project, 2024-01-01, 2024-12-31, -10000.00, Active).

SQL state: 23514

Detail: Failing row contains (3, Invalid Project, 2024-01-01, 2024-12-31, -10000.00, Active).

Query

Query History

1

--Step 4 – Verify data

2

SELECT * FROM project;

3

SELECT * FROM expense;

4

5

Scratch Pad

X

Data Output

Messages

Notifications

Showing rows: 1 to 2

Page No: 1

of 1

	expense_id [PK] integer	project_id integer	expense_date date	amount numeric (10,2)	category text
1	1	1	2024-02-01	3000.00	Labor
2	2	2	2024-05-01	2500.00	Supplies

Query	Query History	Scratch Pad X
<pre> 1 --Step 5 – Error handling example (clean feedback) 2 DO \$\$ 3 BEGIN 4 BEGIN 5 INSERT INTO project (project_name, start_date, end_date, budg 6 VALUES ('Bad Status', '2024-01-01', '2024-06-01', 50000, 'Unk 7 EXCEPTION WHEN others THEN 8 RAISE NOTICE 'Insert failed: %', SQLERRM; 9 END; 10 END; </pre>		
Data Output	Messages	Notifications
NOTICE: Insert failed: new row for relation "project" violates check constraint "project_status_check" DO Query returned successfully in 184 msec.		

B7: E-C-A Trigger for Denormalized Totals (small DML set)

Query	Query History	Scratch Pad X
<pre> 1 --Step 1 – Create sample base tables 2 3 DROP TABLE IF EXISTS expense CASCADE; 4 DROP TABLE IF EXISTS project CASCADE; 5 6 CREATE TABLE project (7 project_id SERIAL PRIMARY KEY, 8 project_name TEXT NOT NULL, 9 total_expense NUMERIC(10,2) DEFAULT 0 10); 11 </pre>		
Data Output	Messages	Notifications
CREATE TABLE Query returned successfully in 290 msec.		
<pre> 1 INSERT INTO project (project_name) VALUES ('Bridge Repair'), ('Hospital 2 </pre>		
Data Output	Messages	Notifications
INSERT 0 2 Query returned successfully in 100 msec.		

Query

Query History

Scratch Pad X

```
1 --Step 2 - Create audit table
2 DROP TABLE IF EXISTS project_audit;
3
4 CREATE TABLE project_audit (
5     audit_id SERIAL PRIMARY KEY,
6     bef_total NUMERIC(10,2),
7     aft_total NUMERIC(10,2),
8     changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
9     key_col TEXT
10 );
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 238 msec.

Query

Query History

Scratch Pad X

```
1 --Step 3 - Create the trigger function
2 CREATE OR REPLACE FUNCTION recompute_project_totals()
3 RETURNS TRIGGER AS $$
4 DECLARE
5     v_bef_total NUMERIC(10,2);
6     v_aft_total NUMERIC(10,2);
7 BEGIN
8     -- Compute the total before change
9     SELECT COALESCE(SUM(amount),0) INTO v_bef_total
10    FROM expense;
11
12     -- Wait for the operation to complete
13     PERFORM pg_sleep(0.1); -- (just for timing clarity in logs)
14
15     -- Update each project's total_expense
16     UPDATE project p
17     SET total_expense = COALESCE((
18         SELECT SUM(e.amount)
19         FROM expense e
20         WHERE e.project_id = p.project_id
21     ),0);
22
23     -- Compute the total after change
24     SELECT COALESCE(SUM(amount),0) INTO v_aft_total
25    FROM expense;
26
27     -- Log change in audit table
28     INSERT INTO project_audit (bef_total, aft_total, key_col)
29     VALUES (v_bef_total, v_aft_total, TG_OP || ' on expense');
30
31     RETURN NULL;
32 END;
33 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 100 msec.

Query

Query History

Scratch Pad X

```

1  --Step 4 – Create the trigger
2  DROP TRIGGER IF EXISTS trg_expense_recompute ON expense;
3
4  CREATE TRIGGER trg_expense_recompute
5  AFTER INSERT OR UPDATE OR DELETE ON expense
6  FOR EACH STATEMENT
7  EXECUTE FUNCTION recompute_project_totals();
8
9
10

```

Data Output

Messages

Notifications

CREATE TRIGGER

Query returned successfully in 140 msec.

No limit

Query

Query History

Scratch Pad X

```

1  --Step 5 – Test the trigger (small mixed DML set)
2
3  -- 1 Insert 2 expenses
4  INSERT INTO expense (project_id, amount) VALUES (1, 1000), (2, 2000);
5
6  -- 2 Update one expense
7  UPDATE expense SET amount = 1500 WHERE expense_id = 1;
8
9  -- 3 Delete one expense

```

Data Output

Messages

Notifications

Showing rows: 1 to 3

Page No: 1 of 1

	audit_id [PK] integer	bef_total numeric (10,2)	aft_total numeric (10,2)	changed_at timestamp without time zone	key_col text
1	1	3000.00	3000.00	2025-10-28 22:50:01.262495	INSERT on expense
2	2	3500.00	3500.00	2025-10-28 22:50:01.262495	UPDATE on expen...
3	3	1500.00	1500.00	2025-10-28 22:50:01.262495	DELETE on expense

B8: Recursive Hierarchy Roll-Up (6–10 rows)

Query

Query History

Scratch Pad X

```

1  --1. Create a table for the hierarchy
2  CREATE TABLE HIER (
3      parent_id INT,
4      child_id INT
5  );
6

```

Data Output

Messages

Notifications

CREATE TABLE

Query returned successfully in 176 msec.

Query

Query History

Scratch Pad

X

```

1 --2. Insert 6-10 rows forming a 3-level hierarchy
2 INSERT INTO HIER (parent_id, child_id) VALUES
3 (1, 2),
4 (1, 3),
5 (2, 4),
6 (2, 5),
7 (3, 6),
8 (4, 7);
9

```

Data Output

Messages

Notifications

INSERT 0 6

Query returned successfully in 111 msec.

Query

Query History

Scratch Pad

X

```

1 --3. Write a recursive WITH query
2 WITH RECURSIVE Hierarchy AS (
3 -- Anchor member: start from root nodes
4 SELECT
5     parent_id AS root_id,
6     child_id,
7     1 AS depth
8 FROM HIER
9 WHERE parent_id NOT IN (SELECT child_id FROM HIER)
10

```

Data Output

Messages

Notifications

Showing rows: 1 to 6

Page No: 1 of 1

	child_id integer	root_id integer	depth integer
1	2	1	1
2	3	1	1
3	4	1	2
4	5	1	2
5	6	1	2
6	7	1	3

Total rows: 6

Query complete 00:00:00.121

CRLF

Ln 24, Col 1

B9: Mini-Knowledge Base with Transitive Inference (≤ 10 facts)

B9

Query

Query History

Scratch Pad X

```

1  --1. Create the Table
2  CREATE TABLE TRIPLE (
3      s VARCHAR(64),
4      p VARCHAR(64),
5      o VARCHAR(64)
6  );
7
8

```

Data Output

Messages

Notifications

CREATE TABLE

Query returned successfully in 282 msec.

Query

Query History

Scratch Pad X

```

4  INSERT INTO TRIPLE VALUES ('Cat', 'isA', 'Mammal');
5  INSERT INTO TRIPLE VALUES ('Mammal', 'isA', 'Animal');
6  INSERT INTO TRIPLE VALUES ('Bird', 'isA', 'Animal');
7  INSERT INTO TRIPLE VALUES ('Animal', 'isA', 'Organism');
8  INSERT INTO TRIPLE VALUES ('Eagle', 'isA', 'Bird');
9  INSERT INTO TRIPLE VALUES ('Whale', 'isA', 'Mammal');
10 INSERT INTO TRIPLE VALUES ('Penguin', 'isA', 'Bird');
11 INSERT INTO TRIPLE VALUES ('Organism', 'isA', 'LivingBeing');
12
13

```

Data Output

Messages

Notifications

INSERT 0 1

Query returned successfully in 138 msec.

Query

Query History

Scratch Pad X

```

1  --Step 3 – Recursive Inference Query (PostgreSQL-compatible)
2  WITH RECURSIVE Inference AS (
3      -- Base facts
4      SELECT s, p, o
5      FROM TRIPLE
6      WHERE p = 'isA'
7
8      UNION ALL
9
10     -- Recursive rule: if A isA B and B isA C, then A isA C
11     SELECT a.s, a.p, b.o
12     FROM Inference a
13     JOIN TRIPLE b ON a.o = b.s
14     WHERE b.p = 'isA'
15 )
16 SELECT DISTINCT s, p, o
17 FROM Inference
18 ORDER BY s, o;

```

Data Output Messages Notifications			
Showing rows: 1 to 29 Page No: 1 of 1			
	s character varying (64)	p character varying (64)	o character varying (64)
1	Animal	isA	LivingBeing
2	Animal	isA	Organism
3	Bird	isA	Animal
4	Bird	isA	LivingBeing
5	Bird	isA	Organism
6	Cat	isA	Animal
7	Cat	isA	LivingBeing
8	Cat	isA	Mammal
9	Cat	isA	Organism
10	Dog	isA	Animal
11	Dog	isA	LivingBeing
12	Dog	isA	Mammal
13	Dog	isA	Organism
14	Eagle	isA	Animal
15	Eagle	isA	Bird
16	Eagle	isA	LivingBeing
17	Eagle	isA	Organism
18	Mammal	isA	Animal
19	Mammal	isA	LivingBeing
20	Mammal	isA	Organism
21	Organism	isA	LivingBeing
22	Penguin	isA	Animal
23	Penguin	isA	Bird
24	Penguin	isA	LivingBeing
25	Penguin	isA	Organism
26	Whale	isA	Animal
27	Whale	isA	LivingBeing
28	Whale	isA	Mammal
29	Whale	isA	Organism

B10: Business Limit Alert (Function + Trigger) (row-budget safe)

Query	Query History	Scratch Pad
1	--Step 1 - Create BUSINESS_LIMITS Table	
2	CREATE TABLE BUSINESS_LIMITS (	
3	rule_key VARCHAR(64),	
4	threshold NUMERIC,	
5	active CHAR(1) CHECK (active IN ('Y', 'N'))	
6	);	
7		
8		
Data Output	Messages	Notifications
CREATE TABLE		
Query returned successfully in 205 msec.		
Query	Query History	Scratch Pad
1	--Seed exactly one active rule:	
2	INSERT INTO BUSINESS_LIMITS (rule_key, threshold, active)	
3	VALUES ('MAX_EXPENSE_LIMIT', 1000, 'Y');	
4		
5		
Data Output	Messages	Notifications
INSERT 0 1		
Query returned successfully in 124 msec.		
Query	Query History	Scratch Pad
1	--Step 2 - Create EXPENSE Table	
2	CREATE TABLE IF NOT EXISTS EXPENSE (	
3	expense_id SERIAL PRIMARY KEY,	
4	description VARCHAR(100),	
5	amount NUMERIC	
6	);	
7		
Data Output	Messages	Notifications
NOTICE: relation "expense" already exists, skipping CREATE TABLE		
Query returned successfully in 80 msec.		
Total rows:	Query complete 00:00:00.080	CRLF Ln 7, Col 1

Query

Query History

Scratch Pad X

```
1  --Step 3 - Create the Function (fn_should_alert)
2  CREATE OR REPLACE FUNCTION fn_should_alert(new_amount NUMERIC)
3  RETURNS INTEGER AS $$
4  DECLARE
5      v_threshold NUMERIC;
6      v_total NUMERIC;
7  BEGIN
8      -- Get active rule threshold
9      SELECT threshold INTO v_threshold
10     FROM BUSINESS_LIMITS
11     WHERE active = 'Y'
12     LIMIT 1;
13
14     -- Calculate total existing expenses
15     SELECT COALESCE(SUM(amount), 0) INTO v_total
16     FROM EXPENSE;
17
18     -- Check if new total would exceed threshold
19     IF v_total + new_amount > v_threshold THEN
20         RETURN 1; -- violation
21     ELSE
22         RETURN 0; -- safe
23     END IF;
24 END;
25 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 127 msec.

Navigation icons: Folder, Download, Edit, Filter, No limit, Play, Stop, Export, Print, Help, Search, List, Question mark.

Query

Query History

Scratch Pad X

```
1  --Step 4 - Create the Trigger
2  CREATE OR REPLACE FUNCTION trg_expense_check()
3  RETURNS TRIGGER AS $$
4  BEGIN
5      IF fn_should_alert(NEW.amount) = 1 THEN
6          RAISE EXCEPTION 'Expense limit exceeded: cannot insert or upd
7      END IF;
8      RETURN NEW;
9  END;
10 $$ LANGUAGE plpgsql;
11
12
13 CREATE TRIGGER expense_limit_trigger
14 BEFORE INSERT OR UPDATE ON EXPENSE
15 FOR EACH ROW
16 EXECUTE FUNCTION trg_expense_check();
17
18
```

Data Output **Messages** Notifications

CREATE TRIGGER

Query returned successfully in 150 msec.

Total rows: Query complete 00:00:00.150

CRLF

Ln 11, Col 1

Query

Query History

Scratch Pad X

1

--Step 5 – Test the System

2

-- Passes (under limit)

3

INSERT INTO EXPENSE (description, amount) VALUES ('Snacks', 200);

4

INSERT INTO EXPENSE (description, amount) VALUES ('Office Supplies', 100);

5

6

-- Fails (over limit)

7

INSERT INTO EXPENSE (description, amount) VALUES ('Laptop', 800);

8

INSERT INTO EXPENSE (description, amount) VALUES ('Team Trip', 1500);

9

10

Data Output

Messages

Notifications

ERROR: Expense limit exceeded: cannot insert or update record
CONTEXT: PL/pgSQL function trg_expense_check() line 4 at RAISE

SQL state: P0001